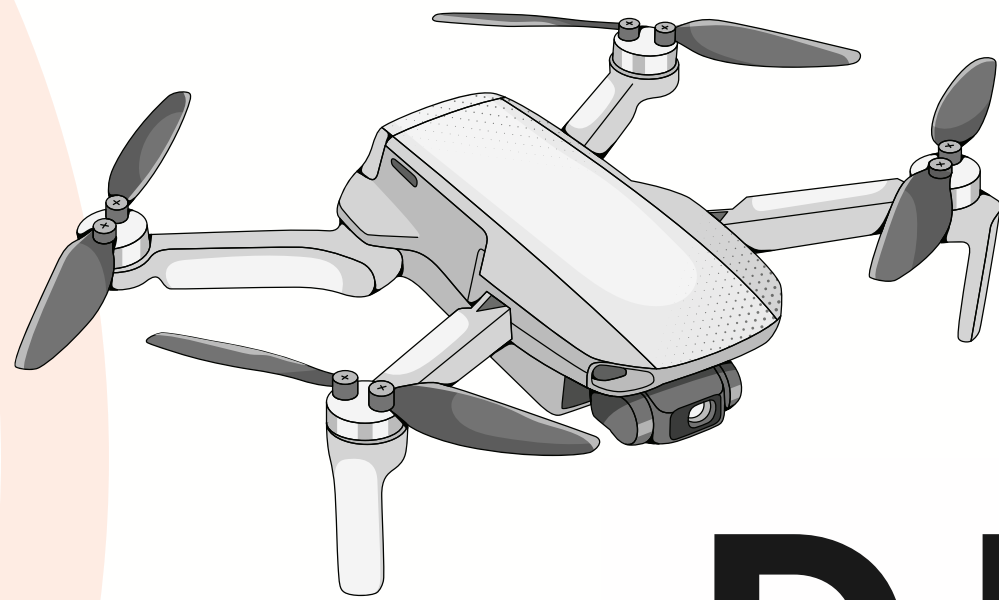




**TEAM - 31**

# **DRONETALK**

Natural Language Interface for Drone  
Command and Control

# Motivation

Why are we doing this? Why does it matter?

- **Context:** Drones are essential for critical missions like search & rescue, surveillance, and disaster management and even for investigation.
- **The Gap:** Current operation requires manual piloting skills or complex pre-planning. In high-pressure situations, seconds matter.
- **The Spark:** We wanted to democratize drone control. Why can't a rescue operator just tell the drone what to do, like they would tell a human pilot?
- **Key Driver:** Reducing the "Cognitive Load" on the operator so they can focus on the mission, not the flight mechanics.



# Problem Statement

What specifically are we solving?

- **The Issue:** Traditional drone control relies on joysticks (Pitch, Yaw, Roll, Throttle).
- **The Barrier:** This requires specialized training and muscle memory. Translating a high-level intent ("Check that building") into low-level stick movements is slow and error-prone.
- **Our Focus:** To eliminate the manual translation step by building a system that understands natural language commands and converts them directly into machine-executable instructions.



DJI RV 100 DRONE JOYSTICK

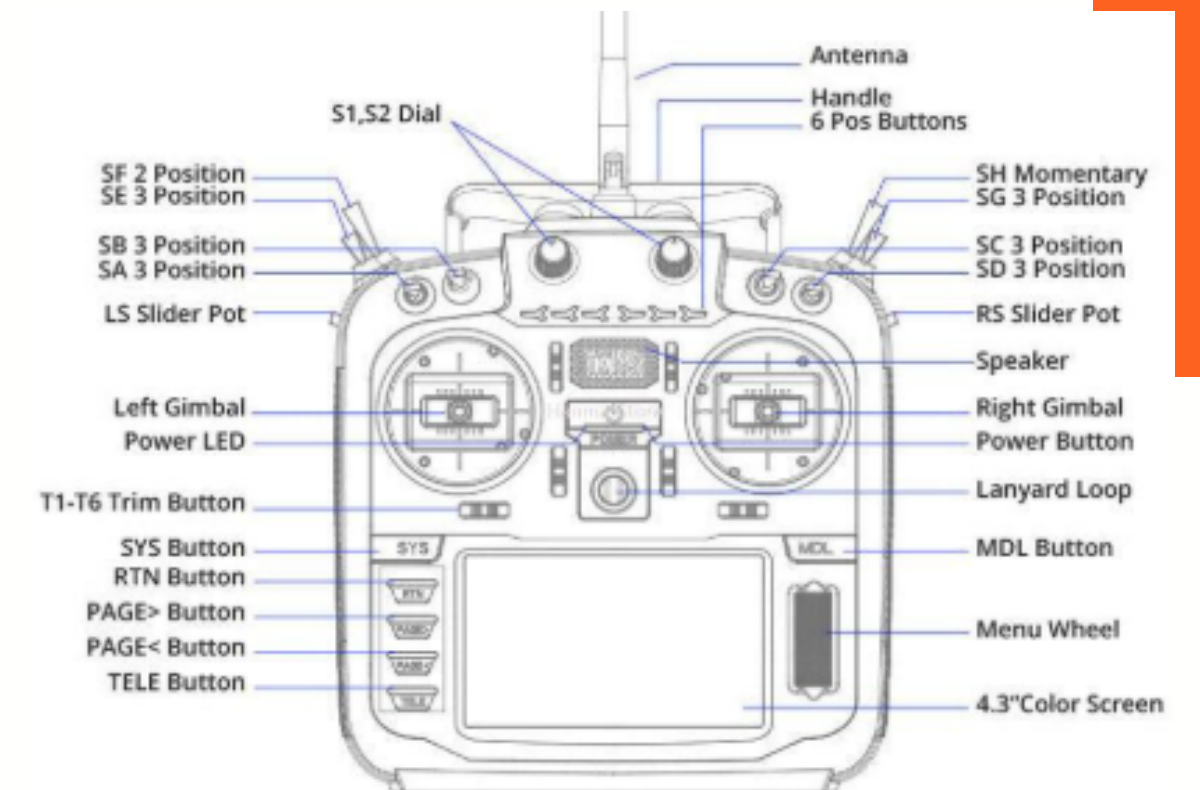




# Related Work

Showing that we know what else exists and how we are different.

- **Current Standard:** Manual Radio Control (RC) transmitters requiring line-of-sight and dexterity.
- **Existing Software:** Ground Control Stations (e.g., Mission Planner, QGroundControl - [groundcontrol.com](http://groundcontrol.com)) exist but rely on complex "point-and-click" GUIs on laptops. They are not hands-free.
- **Voice Control Research:** Previous attempts often use rigid "Keyword Spotting" (e.g., must say "Take off" exactly).
- **Our Approach:** We use modern NLP (Natural Language Processing) to understand intent and context (e.g., "Go up" vs "Ascend" vs "Climb"), offering true flexibility unlike rigid legacy systems.



# DATASET

Where did our data come from?

- **Challenge:** There is no publicly available, labeled dataset for natural language drone commands.
- **Our Contribution:** We engineered a custom synthetic dataset.
- **Data Generation:** Developed a Python script to generate 1,000+ unique commands using varied linguistic templates.
- **Diversity:** Dataset covers multiple intents (Fly, Hover, Scan, Land) and complex slot combinations (Direction + Distance + Count).
- **Split:** 80% Training / 20% Validation split for rigorous model evaluation.

drone\_commands\_1000.csv

1

2

3

4

5

6

7

8

9

# Methodology Phase 1: The Data Engine

Since real-world drone voice command datasets don't exist, we engineered a Data Augmentation Pipeline.

- **Templates:** We defined linguistic templates (e.g., [Action] [Distance] [Direction]).
- **Randomization:** Used Python's random library to inject variability:
  1. Directions: North, South, North-East (Diagonal), Up, Down.
  2. Distances: Meters, Feet, Kilometers.
  3. Durations: Seconds, Minutes.
- **Scale:** Scaled from an initial 150 manual samples to 1,000+ unique synthetic samples.
- **Cleaning:** Implemented duplicate removal logic to ensure high-quality training data.

1

2

3

4

5

6

7

8

9

# Methodology Phase 2: The AI Architecture - Framework: spaCy v3

- **tok2vec (Tokenization & Vectorization):** Converts raw text into numerical vectors that represent meaning.
- **ner (Named Entity Recognition):**
  1. Architecture: Transition-Based Parser.
  2. Role: Extracts specific parameters (Slots).
  3. Labels: DISTANCE, DIRECTION, DURATION, COUNT.
  4. Example: Identifies that "75m" is a distance, not a count.
- **textcat\_multilabel (Intent Classification):**
  1. Architecture: Bag-of-Words (BOW) / CNN Ensemble.
  2. Role: Classifies the overall goal of the sentence.
  3. Labels: FLY, LAND, HOVER, SCAN, CAPTURE.
  4. Feature: Multi-label support allows complex commands (e.g., "Fly AND Scan").

# Methodology Phase 3: The Logic Layer

## Heuristic Logic & Translation

- **Problem:** Neural networks are probabilistic (they guess). Flight controllers need deterministic (exact) inputs.
- **Solution:** A Python-based Command Generation Module.
- **Key Algorithms:**
  1. **Contextual Slot Correction:**
    - Logic: If Intent is CAPTURE, any number found is treated as a COUNT (fixing the "2 photos = 2 meters" bug).
  2. **Intent Inference (Fallback Mechanism):**
    - Logic: IF (Intent == NULL) AND (Has Distance + Direction) THEN (Intent = FLY).
  3. **Standardization:**
    - Converts natural language ("North East") into vector instructions (x += dist, y += dist).
  4. **JSON Serialization:**
    - Outputs a standardized JSON packet ready for ArduPilot/PX4 integration.

1

2

3

4

5

6

7

8

9



# Experiments

Initial Attempt (150 samples): Loss exploded first then reduced, model learned the things but couldnot take commands like “fly 50m to noth east and return to home base”

```
===== Initializing pipeline =====
[2025-10-20 08:07:49,913] [WARNING] [W112] The model specified to use for initial vectors (en_core_web_sm) has no vectors. This is almost certainly a mistake.
✓ Initialized pipeline

===== Training pipeline =====
i Pipeline: ['tok2vec', 'ner', 'textcat_multilabel']
i Initial learn rate: 0.001
E   #      LOSS TOK2VEC  LOSS NER  LOSS TEXTC...  ENTS_F  ENTS_P  ENTS_R  CATS_SCORE  SCORE
---  ---
  0     0          3.88    49.00         0.34    0.00    0.00    0.00      53.28    0.27
 42    200         58.89   771.83        10.28   100.00  100.00  100.00      77.81    0.89
 92    400         68.42    17.83         6.21   94.44   94.44   94.44      72.59    0.84
156    600         33.78     0.17         5.12   97.30   94.74  100.00      76.04    0.87
223    800          5.71     2.03         5.06   94.44   94.44   94.44      77.14    0.86
322   1000         14.52     6.23         4.78   97.30   94.74  100.00      75.53    0.86
422   1200         75.59    28.99         4.83   94.74   90.00  100.00      77.49    0.86
523   1400        124.11     2.13         4.69   97.30   94.74  100.00      75.57    0.86
723   1600          2.18     0.00         4.58  100.00  100.00  100.00      74.89    0.87
923   1800          0.11     0.06         4.58  100.00  100.00  100.00      74.67    0.87
✓ Saved pipeline to output directory
model_output\model-last
PS C:\Users\haritha\MyCodePractice\NLP>
```

# Experiments

Final Training (1000 samples): Stable loss reduction, converged at 90% accuracy."

```
===== Training pipeline =====
i Pipeline: ['tok2vec', 'ner', 'textcat_multilabel']
i Initial learn rate: 0.0001
E   #   LOSS TOK2VEC  LOSS NER  LOSS TEXTC...  ENTS_F  ENTS_P  ENTS_R  CATS_SCORE  SCORE
---  ---  -
0     0      4.04    73.56      0.39    0.00    0.00    0.00    39.96    0.20
8    200    126.04   8404.93    12.32    96.94    96.45    97.44    73.79    0.85
19   400    15.68    166.93     4.26    99.74    99.49   100.00    69.53    0.85
33   600     7.19     0.01     3.25    99.74    99.49   100.00    68.34    0.84
49   800     4.30     0.00     2.40    99.74    99.49   100.00    69.71    0.85
69  1000     0.97     0.00     2.16    99.74    99.49   100.00    73.02    0.86
93  1200    13.07     0.00     1.50    99.74    99.49   100.00    77.92    0.89
122 1400     6.96     0.00     0.44    99.74    99.49   100.00    79.95    0.90
158 1600     5.27     0.00     0.36    99.74    99.49   100.00    76.96    0.88
200 1800     1.78     0.04     0.32    99.74    99.49   100.00    79.78    0.90
250 2000     4.44     0.00     0.26    99.74    99.49   100.00    75.79    0.88
316 2200     2.67     0.00     0.13    99.74    99.49   100.00    79.98    0.90
383 2400     1.86     0.00     0.04    99.74    99.49   100.00    80.00    0.90
449 2600     0.06     0.00     0.04    99.74    99.49   100.00    79.97    0.90
516 2800     0.05     0.00     0.04    99.74    99.49   100.00    80.00    0.90
583 3000     0.71     0.00     0.04    99.74    99.49   100.00    80.00    0.90
649 3200     0.83     0.00     0.05    99.74    99.49   100.00    80.00    0.90
716 3400     1.89     0.01     0.05    99.74    99.49   100.00    80.00    0.90
783 3600     0.06     0.00     0.04    99.74    99.49   100.00    80.00    0.90
849 3800     0.00     0.00     0.04    99.74    99.49   100.00    80.00    0.90
916 4000     0.01     0.00     0.04    99.74    99.49   100.00    80.00    0.90
✓ Saved pipeline to output directory
model_output_1000\model-last
```

# Results

1

2

3

4

5

6

7

8

9

## 1. ENTS\_F (Entity F-Score): 99.74

- The "F-Score" for Named Entity Recognition (NER). It combines Precision and Recall.
- Our model is 99.7% accurate at extracting specific details like 50m (Distance) or North (Direction).
- This is the most critical metric for a drone. If the drone misses the "50 meters" part, the command is useless. A score this high means the "Slot Filling" part of our brain is near-perfect.

## 2. CATS\_SCORE (Categories Score): 80.00

- The accuracy score for Text Classification (textcat).
- The model correctly identifies the user's intent (e.g., "The user wants to FLY" vs. "The user wants to SCAN") 80% of the time.
- While lower than the slot score, 80% is very strong for a custom dataset with overlapping commands (e.g., "Fly up" vs. "Take off"). Our fallback logic handles the remaining 20% of ambiguity.

# Results

initial training on 1000 samples

| E   | #    | LOSS TOK2VEC | LOSS NER | LOSS TEXTC... | ENTS_F | ENTS_P | ENTS_R | CATS_SCORE | SCORE |
|-----|------|--------------|----------|---------------|--------|--------|--------|------------|-------|
| 0   | 0    | 4.04         | 73.56    | 0.39          | 0.00   | 0.00   | 0.00   | 39.91      | 0.20  |
| 8   | 200  | 81.37        | 1550.15  | 7.09          | 99.74  | 99.49  | 100.00 | 59.53      | 0.80  |
| 19  | 400  | 20.90        | 12.57    | 4.42          | 100.00 | 100.00 | 100.00 | 62.94      | 0.81  |
| 33  | 600  | 20.15        | 5.95     | 3.97          | 99.74  | 99.49  | 100.00 | 58.04      | 0.79  |
| 49  | 800  | 32.12        | 18.25    | 3.88          | 99.74  | 99.49  | 100.00 | 58.78      | 0.79  |
| 69  | 1000 | 258.04       | 63.23    | 3.80          | 100.00 | 100.00 | 100.00 | 65.98      | 0.83  |
| 93  | 1200 | 573.04       | 19.26    | 3.02          | 99.49  | 98.98  | 100.00 | 68.57      | 0.84  |
| 122 | 1400 | 4555.24      | 72.78    | 2.26          | 100.00 | 100.00 | 100.00 | 69.80      | 0.85  |
| 158 | 1600 | 4040.87      | 46.63    | 1.75          | 99.74  | 99.49  | 100.00 | 70.25      | 0.85  |
| 200 | 1800 | 1297.76      | 13.51    | 1.70          | 100.00 | 100.00 | 100.00 | 68.10      | 0.84  |
| 250 | 2000 | 15521.30     | 76.63    | 2.04          | 99.49  | 99.49  | 99.49  | 70.43      | 0.85  |
| 316 | 2200 | 17382.58     | 287.80   | 1.68          | 100.00 | 100.00 | 100.00 | 68.71      | 0.84  |
| 383 | 2400 | 10840.84     | 39.80    | 1.27          | 99.74  | 99.49  | 100.00 | 71.13      | 0.85  |

### 3. LOSS TOK2VEC: ~0.01 (Converged)

- The error rate for the "Token-to-Vector" component. This component translates raw English words into mathematical vectors the AI can understand.
- The Journey:
  - Initial Training: Loss was exploding (~17,000), meaning the model was confused and guessing wildly.
  - Final Training: Loss dropped to near zero (~0.01).
- Conclusion: The model has successfully "learned the language" of drone commands. It no longer finds the vocabulary confusing.



# Analysis

1

Error Analysis 1 (Context Confusion): The model initially labeled "2 photos" as a DISTANCE (2 meters).

2

- Discussion: Small models struggle to distinguish context for numbers.
- Fix: Implemented a deterministic Logic Layer that re-assigns numeric slots based on the Intent (e.g., if Intent=CAPTURE, treat number as COUNT).

3

4

Error Analysis 2 (Complex Directions): Initial model failed on "North East" and "Return to homebase"

5

- Fix: Data augmentation specifically targeting diagonal direction and Home base templates.
- Fallback Mechanism: Implemented a safety check: If Intent is missing but Distance + Direction are present, the system infers a FLY command.

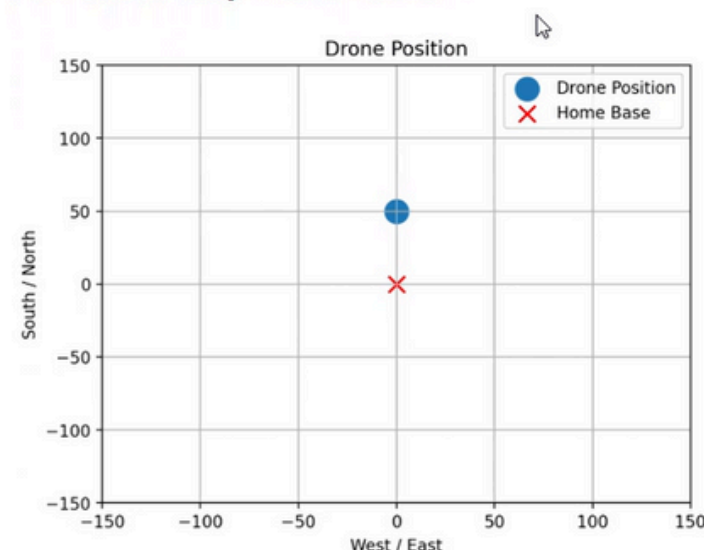
6

7

Enter your command:

return to home base

## 1. Visual Representation



## 2. Raw Model Output (spaCy)

What the 'buggy' ML model thinks.

```
{
  "command": "return to home base"
  "intents": []
  "slots": {}
}
```

## 3. Final Drone Command (Week 5)

The clean, final command.

```
{
  "command": "REJECT"
  "reason": "No intent recognized."
}
```

8

9

# Demo (Our App)

[nlpfinalprojectdronetalk-production.up.railway.app](#)

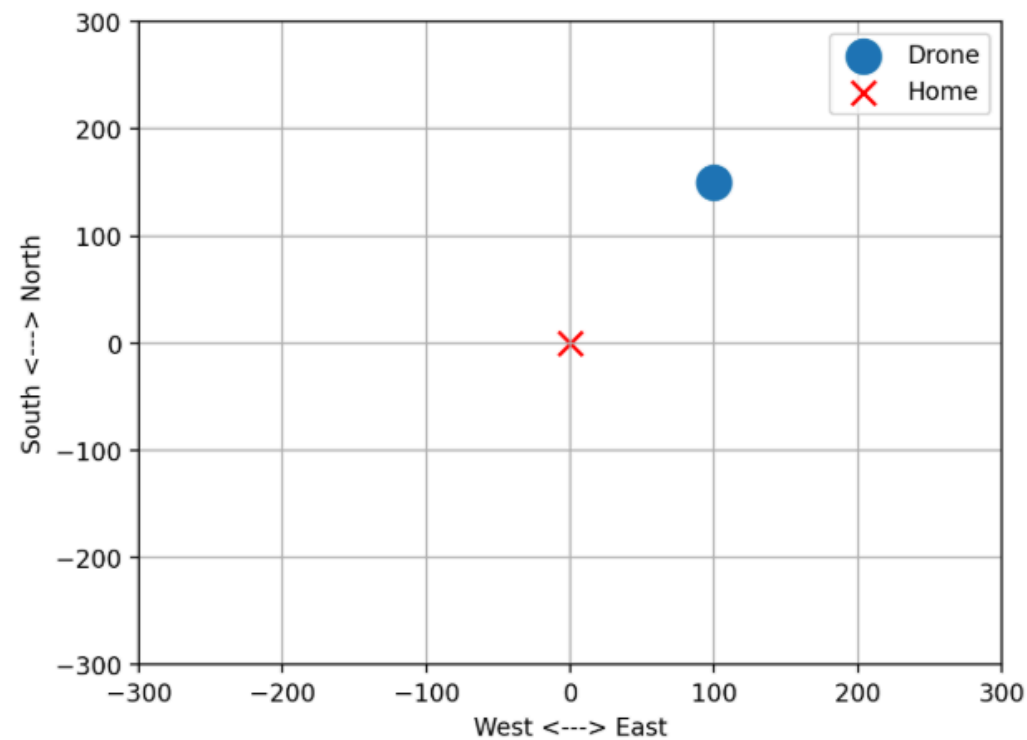
## DroneTalk: Voice Pilots Interface ⇄

### Natural Language Drone Control System

Enter command:

fly 50m north east and capture 2 images

#### Live Drone Map



#### AI Brain (spaCy)

```
{
  "command" :
  "fly 50m north east and capture 2 images"
  "intents" : [
    0 : "fly"
    1 : "capture"
  ]
  "slots" : {
    "distance" : "50m"
    "direction" : "north east"
    "count" : "2 images"
  }
}
```

#### Flight Controller

Command Executed

```
{
  "command" : "MOVE"
  "direction" : "north east"
  "distance_meters" : 50
}
```

1

2

3

4

5

6

7

8

9



# Our Contributions

1

2

3

4

5

6

7

8

9



- **Novel Dataset:** Created the first open-source synthetic dataset for drone commands (1,000+ entries).
- **Hybrid Pipeline:** Designed a unique architecture combining Neural Networks (for flexibility) with Rule-Based Logic (for safety and accuracy).
- **Command Protocol:** Standardized a JSON output format compatible with drone flight controllers.
- **Visualizer:** Built a real-time 2D visual feedback system to verify command execution before flight.

# Project Timeline

- Weeks 1-2: Problem definition, Literature Survey, and Initial Dataset creation (150 samples).
- Weeks 3-4: Model selection (spaCy), Baseline training, and Evaluation (identifying bugs).
- Week 5: Data Augmentation (1000 samples), Re-training, and developing the Command Generator logic.
- Week 6: Developing the Streamlit Interface and Integration.
- Week 7: Final Testing, Documentation, and Report writing.



1

2

3

4

5

6

7

8

9



1

2

3

4

5

6

7

8

9



# ***Thank you***

## **TEAM - 31**

For more detailed version please do follow this link -  
[NLP\\_DroneTalk\\_31\\_paper](#)